

Programación reactiva en el backend

Conclusión

No, el backend no utiliza programación reactiva. Es un backend tradicional **síncrono y bloqueante** basado en Spring MVC + JPA.

Evidencia en `backend/pom.xml` (dependencias)

Dependencia	Rol
spring-boot-starter-web	Modelo servlet tradicional (Tomcat bloqueante).
spring-boot-starter-data-jpa	JPA/Hibernate con JDBC bloqueante.
com.oracle.database.jdbc:ojdbc11	Driver JDBC clásico de Oracle (bloqueante).

Además, **no** se incluyen los starters que habilitarían programación reactiva:

- `spring-boot-starter-webflux`
 - `reactor-core`
 - `spring-boot-starter-data-r2dbc`
-

Evidencia en el controlador (`ProductController.java`)

El controlador devuelve tipos síncronos:

```
@GetMapping
public ResponseEntity<List<ProductResponseDTO>> getAllProducts(...) { ... }

@GetMapping("/{id}")
public ResponseEntity<ProductResponseDTO> getProductById(@PathVariable Long id) { ... }

@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteProduct(@PathVariable Long id) { ... }
```

No se utiliza `Mono<T>` ni `Flux<T>`, que son los tipos reactivos de Project Reactor.

Evidencia en el repositorio (`ProductRepository.java`)

El repositorio extiende `JpaRepository`, que es un repositorio bloqueante:

```
public interface ProductRepository extends JpaRepository<Product, Long> {
    @Query("SELECT p FROM Product p ...")
    List<Product> search(...);
}
```

No se usa `ReactiveCrudRepository` ni `R2dbcRepository`.

Evidencia en el servicio (`ProductServiceImpl.java`)

El servicio usa `@Transactional` y llamadas bloqueantes:

```
@Service
@RequiredArgsConstructor
@Transactional
public class ProductServiceImpl implements ProductService {

    @Override
    public ProductResponseDTO createProduct(ProductRequestDTO request) {
        Product product = Product.builder()...build();
        return toResponseDTO(productRepository.save(product)); // bloqueante
    }

    @Override
```

```
public ProductResponseDTO getProductById(Long id) {  
    return toResponseDTO(findEntityOrThrow(id)); // findById() bloqueante  
}  
  
@Override  
public void deleteProduct(Long id) {  
    Product product = findEntityOrThrow(id);  
    productRepository.delete(product); // bloqueante  
}
```

Aclaración importante

El servicio utiliza `stream()` y `Collectors` para transformar colecciones:

```
return productRepository.search(...)  
    .stream()  
    .map(this::toResponseDTO)  
    .collect(Collectors.toList());
```

Esto es **programación funcional estándar de Java** sobre colecciones, pero **no** es programación reactiva. No hay flujos asíncronos, backpressure ni operadores reactivos.

¿Cómo sería un enfoque reactivo (alternativa)?

Para migrar el backend a programación reactiva se necesitaría:

- `spring-boot-starter-webflux` en lugar de `spring-boot-starter-web`.
- Un driver reactivo como `r2dbc-oracle` en lugar de `ojdbc11`.
- Repositorios reactivos (`ReactiveCrudRepository`) en lugar de `JpaRepository`.
- Hibernate Reactive en lugar de Hibernate clásico.
- Reescribir controladores, servicios y repositorios usando `Mono/Flux`.

Actualmente el proyecto **no** implementa ninguna de estas alternativas.